

# Kubernetes Gateway API + Envoy Gateway + OCI OKE

## Complete Beginner-to-Intermediate Learning Guide

---

### 1. Why This Document Exists

This document summarizes everything discussed about:

- Kubernetes Gateway API
- Envoy Gateway
- OCI OKE networking
- LoadBalancer behavior
- Istio comparison
- AWS ALB Controller comparison
- Private worker nodes with public Load Balancers
- Path-based routing
- Real OCI troubleshooting

The goal is:

- Understand concepts deeply
  - Avoid AWS mental-model confusion
  - Build a future-proof ingress architecture
  - Learn production-grade Kubernetes networking fundamentals
- 

### 2. The Biggest Beginner Confusion

Most beginners think:

Gateway API itself exposes applications

This is WRONG.

Gateway API is only:

A standard + language + API contract

It does NOTHING by itself.

Exactly like:

- Terraform code alone does not create infrastructure
- Kubernetes YAML alone does not run networking
- Dockerfile alone does not run containers

Something must IMPLEMENT it.

---

## 3. Core Components Explained Simply

### 3.1 Gateway API

Gateway API is a Kubernetes networking standard.

It defines resources like:

- GatewayClass
- Gateway
- HTTPRoute

It is NOT a proxy. It is NOT a load balancer. It is NOT a controller.

Think of it as:

A traffic rulebook

---

### 3.2 Envoy Gateway

Envoy Gateway is the controller implementation.

It:

- Watches Gateway API objects
- Configures Envoy proxies
- Creates Kubernetes Services
- Implements routing logic

It does NOT directly create OCI cloud resources.

Think of it as:

The traffic management company

### 3.3 OCI Cloud Controller Manager (CCM)

OCI CCM is responsible for cloud integration.

It:

- Talks to OCI APIs
- Creates OCI Load Balancers
- Attaches public IPs
- Configures backend sets

This is the REAL cloud integration layer.

Think of it as:

The construction workers that build roads and entrances

### 3.4 GatewayClass

GatewayClass tells Kubernetes:

Which controller should handle traffic

Example:

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: envoy
spec:
  controllerName: gateway.envoyproxy.io/gatewayclass-controller
```

Equivalent concept:

```
StorageClass -> for storage
GatewayClass -> for ingress traffic
```

---

## 3.5 Gateway

Gateway defines:

- Entry point
- Ports
- Protocols
- Public/private exposure

This is the MOST IMPORTANT resource.

When created:

```
Gateway
-> Envoy Gateway reacts
-> Creates Service type=LoadBalancer
-> OCI CCM creates OCI Load Balancer
```

Example:

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: public-gateway
spec:
  gatewayClassName: envoy
  listeners:
  - name: http
    port: 80
    protocol: HTTP
```

---

## 3.6 HTTPRoute

HTTPRoute handles routing.

It decides:

- /api -> backend A
- /web -> backend B
- header matching
- canary routing
- weighted traffic

It DOES NOT expose applications to the internet.

It only routes traffic AFTER it enters.

Example:

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-routing
spec:
  parentRefs:
  - name: public-gateway
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /web
    backendRefs:
    - name: web
      port: 80
```

---

## 4. Full Traffic Flow (Most Important Section)

### Final Architecture

```
Internet
  ↓
OCI Public Load Balancer
  ↓
Envoy Gateway Pods
  ↓
HTTPRoute Rules
  ↓
Kubernetes Services
```

↓  
Pods

Detailed breakdown:

```
Gateway API YAML
  ↓
Envoy Gateway Controller
  ↓
Creates Service type=LoadBalancer
  ↓
OCI Cloud Controller Manager
  ↓
OCI creates public Load Balancer
  ↓
Traffic enters cluster
```

## 5. OCI Networking Architecture

### Production-Recommended Design

Worker nodes should remain PRIVATE.

This is correct production architecture.

Architecture:

```
Public Subnet
  -> OCI Load Balancer

Private Subnet
  -> Kubernetes worker nodes
  -> Pods
```

Internet traffic:

```
Internet
  ↓
Public OCI LB
  ↓
Private worker nodes
```

↓  
Pods

Nodes DO NOT require public IPs.

---

## 6. Understanding OKE Subnets

The following subnets existed in the environment:

### 6.1 Worker Node Subnet

Example:

```
sub-ng-oke1-*
```

Purpose:

- Worker nodes
  - Usually private
- 

### 6.2 Pod Subnet

Example:

```
sub-ng-pod-*
```

Purpose:

- Pod IP allocation
- 

### 6.3 Kubernetes API Endpoint Subnet

Example:

```
sub-k8sApiEndpoint-*
```

Purpose:

- Kubernetes API access
- 

## 6.4 Service Load Balancer Subnet (IMPORTANT)

Example:

```
sub-svclb-oke1-prime360novac-1
```

Purpose:

```
Public OCI Load Balancers for Kubernetes Services
```

THIS subnet must be used for Gateway API public exposure.

---

## 7. Why Service type=LoadBalancer Initially Failed

Initial behavior:

```
EXTERNAL-IP = <pending>
```

Reason:

OKE does NOT automatically choose the LB subnet.

Unlike AWS:

```
OCI requires explicit subnet annotation
```

---

## 8. Required OCI Annotation

Correct annotation:



```
service.beta.kubernetes.io/oci-load-balancer-subnet1: <SUBNET_OCID>
```

Actual OCID used:

```
ocid1.subnet.oc1.ap-  
mumbai-1.aaaaaaatdk6s3v5zvw4cylvy4vam2s4heo4diqgjczbcstaqklstfbrjl3q
```

Without this annotation:

```
LoadBalancer stays pending forever
```

---

## 9. Successful Validation

Validation steps:

```
kubectl create deployment test --image=nginx
```

Initial expose command:

```
kubectl expose deployment test --type=LoadBalancer --port=80
```

Result:

```
EXTERNAL-IP: <pending>
```

Then recreated Service with OCI subnet annotation.

Final result:

```
EXTERNAL-IP: 80.225.249.255
```

This proved:

- OCI Load Balancer integration works
- Private worker nodes are fine
- Public LB subnet works

- IAM is correct
  - Gateway API will work
- 

## 10. AWS vs OCI Mental Model Correction

### AWS Load Balancer Controller

AWS combines:

- Cloud integration
- ALB creation
- Routing logic

into ONE controller.

This hides complexity.

Architecture:

```
graph TD; Ingress --> AWS_Load_Balancer_Controller[AWS Load Balancer Controller]; AWS_Load_Balancer_Controller --> ALB_created[ALB created];
```

---

### OCI + Gateway API

OCI uses separated responsibilities.

Architecture:

```
graph TD; Gateway_API[Gateway API] --> Envoy_Gateway[Envoy Gateway]; Envoy_Gateway --> Service_type_LoadBalancer[Service type=LoadBalancer]; Service_type_LoadBalancer --> OCI_CCM[OCI CCM]; OCI_CCM --> OCI_Load_Balancer[OCI Load Balancer];
```

This is more Kubernetes-native and future-proof.

---

## 11. Istio vs Envoy Gateway

### 11.1 Istio

Istio is a FULL service mesh.

It handles:

- North-south traffic
- East-west traffic
- mTLS
- Sidecars
- Internal security
- Service-to-service observability

Pros:

- Extremely powerful
- Enterprise-grade
- Strong security

Cons:

- Complex
  - Heavy
  - Large learning curve
  - Operationally expensive
- 

### 11.2 Envoy Gateway

Envoy Gateway focuses ONLY on ingress.

It handles:

- External exposure
- Path-based routing
- Traffic policies
- TLS
- Gateway API

Pros:

- Lightweight
- Easier to learn
- Gateway API-native
- Future-proof

Cons:

- Not a full mesh
  - No east-west traffic management
- 

## 11.3 Decision Guidance

Use Istio if you need:

- mTLS between pods
- Internal service mesh
- Zero-trust networking
- East-west traffic control

Use Envoy Gateway if you need:

- Public ingress
  - API exposure
  - Path routing
  - Simpler operations
  - Lower resource usage
- 

## 12. Why Envoy Gateway Was Recommended

Reasons:

- Already familiar with Istio concepts
- Wants future-proof architecture
- Wants advanced routing
- Wants Gateway API-native design
- OCI setup is ingress-focused
- Does not currently require service mesh

Recommendation:

## 13. All Required YAML Files

### 13.1 GatewayClass

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: envoy
spec:
  controllerName: gateway.envoyproxy.io/gatewayclass-controller
```

---

### 13.2 Gateway

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: public-gateway
  namespace: default
spec:
  gatewayClassName: envoy
  listeners:
  - name: http
    port: 80
    protocol: HTTP
```

---

### 13.3 HTTPRoute

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-routing
spec:
  parentRefs:
  - name: public-gateway
  rules:
```

```
- matches:
  - path:
      type: PathPrefix
      value: /web
    backendRefs:
      - name: web
        port: 80

- matches:
  - path:
      type: PathPrefix
      value: /api
    backendRefs:
      - name: api
        port: 5678
```

---

## 14. Installing Envoy Gateway

### Install Gateway API CRDs

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.0.0/standard-install.yaml
```

---

### Install Envoy Gateway

```
helm repo add envoy-gateway https://envoyproxy.github.io/gateway-helm
helm repo update
```

Install:

```
helm install envoy-gateway envoy-gateway/envoy-gateway
  -n envoy-gateway-system
  --create-namespace
  --set service.annotations."service\beta\kubernetes\io/oci-load-balancer-
subnet1"=ocid1.subnet.oc1.ap-
mumbai-1.aaaaaaaaatdk6s3v5zvw4cylvy4vam2s4heo4diqgjczbcstaqklstfbrjl3q
```

# 15. Important Beginner Corrections

## Wrong Assumption 1

Gateway API creates Load Balancers

Correct:

OCI CCM creates Load Balancers

---

## Wrong Assumption 2

Envoy Gateway talks to OCI APIs

Correct:

Envoy Gateway only creates Kubernetes Services

---

## Wrong Assumption 3

Worker nodes must be public

Correct:

Worker nodes should remain private

---

## Wrong Assumption 4

Gateway API replaces service mesh

Correct:

Gateway API focuses mainly on ingress traffic

## 16. Final Simplified Mental Model

### Simplest possible understanding

Gateway API = language  
Envoy Gateway = traffic engine  
Gateway = entrance  
HTTPRoute = directions  
OCI LB = actual internet-facing door

## 17. Final Architecture Summary

```
Internet
  ↓
OCI Public Load Balancer
  ↓
Envoy Gateway
  ↓
Gateway API Routing Rules
  ↓
Kubernetes Services
  ↓
Pods
```

## 18. Production Learning Path

Recommended next topics:

1. HTTPS with cert-manager
2. TLS termination
3. Weighted routing / canary
4. Retry and timeout policies
5. Authentication
6. Web Application Firewall (WAF)



7. Rate limiting
  8. Private Gateways
  9. Multi-Gateway architecture
  10. Gateway API policies
  11. Observability with Prometheus + Grafana
  12. Migrating Istio VirtualService -> HTTPRoute
- 

## 19. Final Takeaway

AWS made ingress look simple because:

ALB controller combined everything

Gateway API separates concerns correctly:

Gateway API -> intent  
Envoy Gateway -> routing  
OCI CCM -> cloud resources  
OCI LB -> actual infrastructure

This architecture is:

- More modular
  - More portable
  - More Kubernetes-native
  - More future-proof
- 

## END